

```

    } else {
        // RECURSIVE CASE
        ❷ shortestWithBaseCase(false);
        document.write("Returning from recursive case.<br />");
        return;
    }
}

document.write("Calling shortestWithBaseCase(false):<br />");
❸ shortestWithBaseCase(false);
document.write("<br />");
document.write("Calling shortestWithBaseCase(true):<br />");
❹ shortestWithBaseCase(true);
</script>

```

When you run this code, the output looks like this:

```

Calling shortestWithBaseCase(False):
shortestWithBaseCase(False) called.
Returning from base case.

```

```

Calling shortestWithBaseCase(True):
shortestWithBaseCase(True) called.
shortestWithBaseCase(False) called.
Returning from base case.
Returning from recursive case.

```

This function doesn't do anything useful except provide a short example of recursion (and it could be made shorter by removing the text output, but the text is useful for our explanation). When `shortestWithBaseCase(False)` is called ❸, the base case is executed and the function merely returns ❶. However, when `shortestWithBaseCase(True)` is called ❹, the recursive case is executed and `shortestWithBaseCase(False)` is called ❷.

It's important to note that when `shortestWithBaseCase(False)` is recursively called from ❷ and then returns, the execution doesn't immediately move back to the original function call at ❹. The rest of the code in the recursive case after the recursive call still runs, which is why `Returning from recursive case.` appears in the output. `Returning from the base case` doesn't immediately return from all the recursive calls that happened before it. This will be important to keep in mind in the `countDownAndUp()` example in the next section.

Code Before and After the Recursive Call

The code in a recursive case can be split into two parts: the code before the recursive call and the code after the recursive call. (If there are two recursive calls in the recursive case, such as with the Fibonacci sequence example in [Chapter 2](#), there will be a before, a between, and an after. But let's keep it simple for now.)

The important thing to know is that reaching the base case doesn't necessarily mean reaching the end of the recursive algorithm. It only means the base case won't continue to make recursive calls.